SmartHealth AI Assistant

Team Members

Name	Roll No.	Role
Saad Abrar	22I-1238	Project Manager & Frontend Dev
Umair Nawaz	22I-0913	Backend Developer & DevOps Eng.
Hamza Khurram	21I-0735	AI/ML Engineer & QA Lead

Course: Software Project Management (SPM)
Session: Spring 2026
Institute: FAST NUCES, Islamabad
Date: May 12, 2026

Contents

Chapter 1

Executive Summary

Project at a Glance

Project Name: SmartHealth AI Assistant **Type:** Full-Stack Web Application with AI/ML

Duration: 16 weeks (Spring 2026) **Methodology:** Agile Scrum (8 × 2-week sprints)

SmartHealth AI Assistant is a full-stack web application that modernises healthcare service delivery through artificial intelligence. The platform rests on three pillars: comprehensive patient health record management, AI-driven symptom analysis using Natural Language Processing (NLP) and machine learning, and a modern architecture spanning React, Node.js/Express, PostgreSQL, and Python microservices.

The system addresses persistent healthcare challenges—delayed diagnosis from poor symptom articulation, inefficient appointment scheduling, and fragmented health records—by offering intelligent symptom analysis, automated specialist recommendations, and streamlined booking, thereby improving both patient experience and provider efficiency.

Beyond its technical scope, the project provides a structured environment for applying Software Project Management (SPM) concepts including Agile/Scrum, risk management, quality assurance, resource planning, and team collaboration.

Chapter 2

Project Overview

2.1 Problem Statement

Healthcare systems globally face compounding challenges:

- Patients often struggle to articulate symptoms accurately, leading to delayed or misdirected care.
- Appointment scheduling workflows are slow and error-prone.
- Medical specialists frequently receive inappropriate referrals due to insufficient initial assessment.

These gaps result in overcrowded emergency departments, poor resource utilisation, and delayed treatment.

2.2 Proposed Solution

SmartHealth AI Assistant delivers an integrated digital health platform with the following capabilities:

2.3 Project Objectives

1. Develop a responsive web application with an intuitive interface for symptom reporting and health management.
2. Implement NLP-based symptom extraction achieving $> 85\%$ accuracy on a validation dataset.
3. Build machine learning models for disease prediction using Random Forest and Neural Network approaches.
4. Deliver a secure RESTful API backend with JWT authentication and role-based access control.
5. Integrate a real-time notification system for appointment confirmations and reminders.

Feature	Description
AI-Powered Symptom Analysis	Natural language interface processing patient-described symptoms, performing contextual follow-up, and producing severity assessments.
Disease Prediction Engine	ML models correlating symptoms with probable conditions, returning ranked predictions with confidence scores.
Intelligent Specialist Matching	Automated routing of patients to appropriate specialists based on predicted conditions.
Appointment Management	Real-time booking with availability checking, confirmations, reminders, and full history.
Health Data Dashboard	Personalised portal for tracking health metrics, symptom trends, and PDF health report exports.

Table 2.1: Core Feature Set of SmartHealth AI Assistant

6. Apply SPM principles—Agile/Scrum, risk management, and quality assurance—throughout the development lifecycle.

Chapter 3

Technical Architecture

3.1 Technology Stack

Layer	Technologies
Frontend	React 18 + TypeScript, Redux Toolkit, Material-UI, Chart.js, Axios
Backend	Node.js / Express.js, JWT, Sequelize ORM, Swagger, Socket.io
AI / ML Services	Python 3.11, FastAPI, spaCy, NLTK, BERT transformers, scikit-learn, TensorFlow
Database & Storage	PostgreSQL 15, Redis, Cloud file storage
DevOps	Git / GitHub, Docker, GitHub Actions CI/CD, Jest, Pytest, ESLint, Prettier

Table 3.1: Full Technology Stack

3.2 System Architecture

The system follows a **microservices architecture** organised into three layers:

3.2.1 Presentation Layer

A React single-page application providing interfaces for symptom input, health dashboards, appointment booking, and administrative functions.

3.2.2 Application Layer

An Express.js API Gateway handles authentication, request validation, and service routing. Business logic services manage user accounts, appointments, notifications, and health records. Dedicated Python FastAPI microservices handle all AI/ML operations—symptom analysis, disease prediction, and specialist recommendations.

3.2.3 Data Layer

PostgreSQL stores all persistent data including user profiles, medical histories, appointments, and prediction logs. Redis optimises response times and manages session data. File storage maintains medical documents and trained model files.

External integrations include email/SMS services, payment gateways, and analytics platforms.

Chapter 4

Core Features

4.1 User Management

Secure registration and authentication with email verification, role-based access control (Patient / Doctor / Admin), full profile and medical history management, and password recovery.

4.2 AI-Powered Symptom Analysis

A free-text input interface enables patients to describe symptoms naturally. Real-time NLP processes and normalises the input, generates contextual follow-up questions, performs severity assessment, and runs multi-symptom correlation analysis.

4.3 Disease Prediction Engine

Probabilistic ML models trained on curated medical datasets return ranked disease recommendations with confidence intervals and plain-language explanations. A feedback loop enables continuous model improvement.

4.4 Appointment System

Real-time doctor availability checking, calendar-based booking, automated email/SMS confirmations and reminders, rescheduling and cancellation support, full appointment history, and a doctor rating system.

4.5 Health Dashboard

Patients can view a timeline of health events, monitor symptom trends, track upcoming and past appointments, export PDF health reports, and receive personalised health insights.

Chapter 5

Software Project Management Approach

5.1 Development Methodology

The project uses **Agile Scrum** with 2-week sprints across a 16-week semester.

- **Sprint Structure:** 8 sprints, each opened with sprint planning and closed with review and retrospective.
- **Daily Standups:** 15-minute synchronisation to surface blockers and track progress.
- **Backlog Management:** Prioritised product backlog with user stories, acceptance criteria, and story points.
- **Version Control:** Git flow branching with feature branches, pull requests, and peer code reviews.
- **Documentation:** Continuous documentation using Markdown, Swagger API references, and meeting minutes.

5.2 Project Timeline

5.3 Team Roles and Responsibilities

All team members participate in code reviews, documentation, and cross-functional collaboration.

Weeks	Phase	Key Deliverables
1–2	Initiation	Requirements analysis, feasibility study, project charter, tool setup
3–4	Planning	Architecture design, database schema, API specifications, sprint backlog
5–6	Development I	User authentication, core UI, database setup, API scaffolding
7–8	Development II	Symptom input module, NLP integration, initial ML model
9–10	Development III	Disease prediction, specialist matching, appointment system
11–12	Development IV	Dashboard, notifications, doctor profiles, service integration
13–14	Testing	Unit, integration, and UAT testing; bug resolution
15–16	Deployment	Final deployment, documentation completion, project presentation

Table 5.1: 16-Week Project Timeline

Name	Roll No.	Responsibilities
Saad Abrar	22I-1238	Project Manager & Frontend Developer: Overall coordination, stakeholder communication, React UI development, and responsive design.
Umair Nawaz	22I-0913	Backend Developer & DevOps Engineer: Express.js API development, database design, CI/CD pipeline setup, deployment, and monitoring.
Hamza Khurram	21I-0735	AI/ML Engineer & QA Lead: NLP and machine learning model development, testing strategy, and quality assurance.

Table 5.2: Team Composition and Role Assignments

Chapter 6

Risk Management

Risk	Impact	Mitigation Strategy
AI model accuracy below threshold	**High**	Evaluate multiple algorithms, conduct extensive testing, implement rule-based fallbacks
Team member unavailability	**Medium**	Cross-train all members, maintain thorough documentation, keep plans flexible
Scope creep	**High**	Enforce a change control process with documented stakeholder sign-off
Security vulnerabilities	**Critical**	Regular security audits, data encryption at rest and in transit, penetration testing
Integration challenges	**Medium**	Prioritise early integration, define explicit API contracts, apply contract testing
Technology learning curve	**Medium**	Dedicated onboarding sessions, pair programming, proof-of-concept exercises

Table 6.1: Risk Register with Mitigation Strategies

Chapter 7

Quality Assurance

7.1 Testing Strategy

- **Unit Testing:** Frontend with Jest and React Testing Library (80% coverage target); Backend with Mocha and Chai (85% target); AI services with Pytest (75% target).
- **Integration Testing:** API endpoints via Supertest, database integration validation, and AI model integration tests.
- **End-to-End Testing:** Cypress automation covering critical user flows—registration, symptom submission, and appointment booking.
- **Performance Testing:** Load testing with K6; API response time monitoring (target: <200 ms); database query optimisation.
- **Security Testing:** OWASP Top 10 vulnerability scanning, static code analysis with Snyk, and penetration testing.

7.2 Quality Metrics

Metric	Target
Code coverage (critical modules)	$\geq 80\%$
Bug density	< 1 defect / 100 LoC
API response time (95th percentile)	< 200 ms
System uptime (demo period)	$\geq 99\%$
AI model accuracy (validation set)	$> 85\%$

Table 7.1: Defined Quality Metrics

Chapter 8

Expected Outcomes and Deliverables

8.1 Technical Deliverables

- Fully functional web application deployed and publicly accessible.
- Source code repository with comprehensive commit history and inline documentation.
- Trained AI/ML models with evaluation reports and performance benchmarks.
- Complete API reference using Swagger/OpenAPI.
- Automated test suites integrated into the CI/CD pipeline.
- Docker containerisation for all services.
- Database schema with migration scripts.

8.2 Documentation Deliverables

Software Requirements Specification (SRS), System Design Document (SDD), Database Design Document, API Reference Guide, User Manual, Developer Setup Guide, Testing Documentation, Deployment Guide, and Final Project Report.

8.3 Project Management Deliverables

Project Charter, Work Breakdown Structure (WBS), Gantt Chart, Risk Management Plan, Sprint Reports (8 total), Meeting Minutes, and Final Presentation slides.

Chapter 9

Success Criteria

The project will be considered successful when **all** of the following conditions are met:

1. All core features are implemented, functional, and covered by tests.
2. AI symptom analysis achieves a minimum of 85% accuracy on the validation dataset.
3. The application passes security audits and meets defined performance benchmarks.
4. The system is deployed with 99% uptime sustained during the demonstration period.
5. User acceptance testing yields a satisfaction rating above 80%.
6. Code quality metrics meet defined standards across all modules.
7. All technical and project management documentation is delivered in full.
8. SPM concepts are applied and evidenced throughout the project lifecycle.
9. The project is completed within the 16-week timeline and budget constraints.
10. A live demonstration is successfully delivered to stakeholders.

Chapter 10

Conclusion

SmartHealth AI Assistant is a comprehensive software engineering project that unifies data management, artificial intelligence, and full-stack web development while providing a structured environment for applying SPM principles. It addresses real-world healthcare challenges through evidence-based technology decisions and follows industry-standard development practices.

Through this project, the team will build hands-on experience in modern web technologies, AI/ML implementation, database design, API development, and project management methodologies—including Agile/Scrum, risk management, and quality assurance. The resulting deliverables will demonstrate both technical depth and management proficiency, serving as a meaningful portfolio piece for each team member.

With a clearly structured 16-week timeline, defined milestones, and rigorous quality standards, the project scope is ambitious yet achievable—positioning the team for a strong final demonstration and a solid foundation for professional software development careers.